

AuthPortal Project Overview

This document explains the full-stack authentication project I built using Spring Boot, React, PostgreSQL, JWT cookies, OTP verification, password recovery, role-based access, and account/profile management features.

PROJECT TYPE

Individual Full-Stack Project

BACKEND

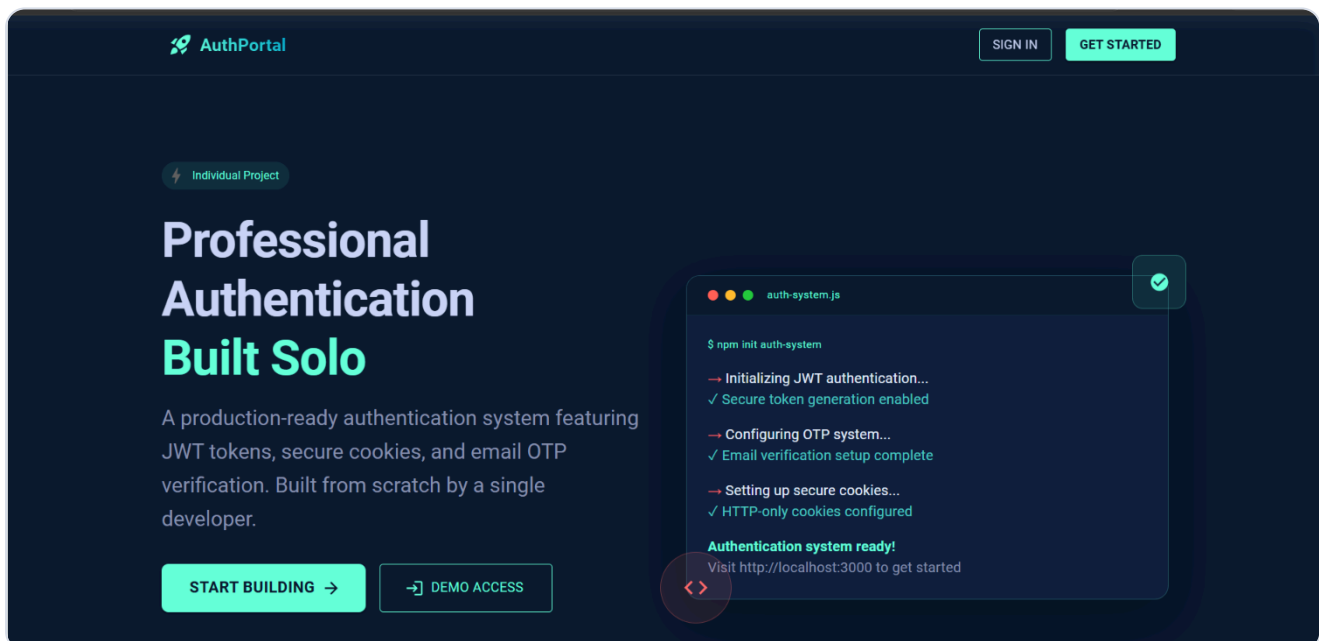
Spring Boot / Java 21

FRONTEND

React / Vite / Material UI

UI Preview

The landing page presents the authentication system as a portfolio-style entry point. It links users to signup and login, and it introduces the main idea of the project without exposing backend details directly in the UI.

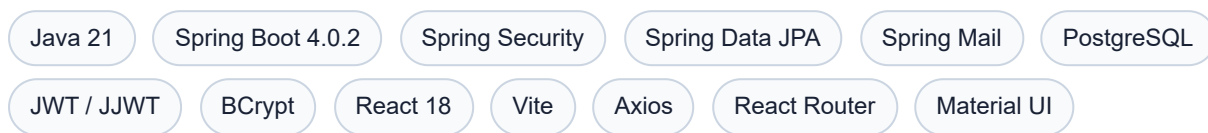


What the Project Does

AuthPortal handles common account flows that a real application normally needs: registration, email verification, login, protected user/admin endpoints, profile viewing, password changes, forgot-password recovery, account deletion, and media uploads.

The project is REST API based. I did not find WebSocket, Socket.IO, or SSE code in the current codebase, so real-time messaging or real-time notifications are not part of this version.

Technology Stack



System Flow

User

- > React frontend
- > Axios request with credentials
- > Spring Boot REST API
- > JWTAuthFilter and Spring Security
- > Controller
- > Service
- > Repository
- > PostgreSQL

Backend Implementation

Controllers

The controller layer exposes API endpoints for authentication, forgot-password recovery, user account actions, admin dashboard access, and media-related user requests.

Services

The service layer contains the main business logic, including signup, login, OTP verification, resend controls, password update logic, account deletion, and image storage handling.

Repositories

Spring Data JPA repositories are used for database access. The backend queries users by email, phone number, refresh token, and forgot-password relation.

Security

Spring Security is configured as stateless. JWT tokens are validated by a custom filter and roles are enforced through security rules and method-level authorization.

Authentication Details

Registration starts by saving an unverified user. The backend generates an OTP, sends it through email, and keeps the email in an HttpOnly cookie to support the verification and resend flow. Login is only accepted after verification.

After login, the backend creates JWT access and refresh tokens. The tokens are added as HttpOnly cookies, which keeps JavaScript from directly reading them and allows the frontend to send authenticated requests using `withCredentials`.

Token / OTP	Usage	Lifetime in Code
ACCESS	Protected API access	25 minutes
REFRESH	Stored during login for longer session support	7 days
VERIFY	Forgot-password password reset permission	30 minutes
Registration OTP	New account verification	2 minutes
Forgot-password OTP	Recovery verification	5 minutes

Forgot Password and OTP Recovery

The recovery flow is more advanced than a simple email-only reset. The user has to provide at least two identifiers from email, backup email, and phone number. The backend matches those details against stored user data before sending the OTP.

```
Recovery details
-> Match at least two identifiers
-> Create or update ForgotPassword record
-> Send OTP by selected channel
-> Verify OTP
-> Create VERIFY token cookie
-> Change password with BCrypt hashing
-> Remove VERIFY token and cleanup recovery record
```

Database Design

The database model is centered around the `User` entity and a one-to-one `ForgotPassword` entity. The user table stores identity fields, password hash, role, verification state, resend control fields, refresh token, temporary email, and media paths. Forgot-password data is kept separately so recovery state can be managed without overloading the user row.

User 1 ----- 1 ForgotPassword

Frontend Implementation

The React frontend contains separate pages for landing, signup, OTP verification, login, forgot password, home dashboard, profile, settings, and admin dashboard. API calls are made through a shared Axios instance configured with credentials, so cookies from the backend are included in requests.

Most frontend state is local component state. I did not use Redux or a separate global state library in this version. The UI uses Material UI components, icons, dialogs, forms, tabs, cards, drawers, and responsive grid layouts.

File Upload and Media Handling

Profile and cover image uploads are sent as multipart form data. The backend validates file presence, size, and content type before saving the file under the configured media root. Stored public paths are saved in the user record and served through the `/media/**`` resource handler.

Important API Groups

Group	Endpoints	Purpose
Auth	<code>/auth/register</code> , <code>/auth/login</code> , <code>/auth/verify-code</code> , <code>/auth/resend-otp</code> , <code>/auth/logout</code>	Signup, login, verification, OTP resend, and cookie cleanup
Forgot Password	<code>/forgotpass/send-otp</code> , <code>/forgotpass/resend-otp</code> , <code>/forgotpass/verify-otp</code> , <code>/forgotpass/change-password</code>	Account recovery and password reset
User	<code>/user/me</code> , <code>/user/update-name</code> , <code>/user/update-email</code> , <code>/user/update-password</code> , <code>/user/delete</code>	Authenticated profile and account management
Media	<code>/user/me/profile-photo</code> , <code>/user/me/cover-photo</code> , <code>/media/**`</code>	Upload and serve profile/cover images
Admin	<code>/admin/dashboard</code>	Admin-only dashboard summary

Security and Configuration Notes

The project uses environment-variable based configuration for database credentials, email credentials, JWT secret, cookie settings, and media root path. Real passwords, app passwords, JWT secrets, and API keys should not be committed to GitHub.

If a sensitive local file is already tracked by Git, remove only the Git tracking with ``git rm -cached path/to/file``, then add that path to ``.gitignore``. This does not delete the local file.

What I Learned

This project helped me understand authentication as a full workflow instead of only login and register. I worked through JWT cookies, OTP verification, recovery logic, role-based access, upload validation, database relationships, frontend API integration, and safe configuration handling.

Future Improvements

- Add backend tests for authentication, OTP, recovery, and upload flows.
- Add a dedicated frontend protected-route wrapper.
- Improve refresh-token rotation and server-side invalidation.
- Align the profile edit dialog with existing update endpoints.
- Add deployment-ready production configuration.